Лабораторные работы «Организация ЭВМ и систем»

Лабораторная работа №5

Изучение машинных инструкций процессора

Цель работы

Познакомиться с машинными инструкциями процессора.

Оборудование

- Персональный компьютер

- Компилятор C/C++

- Средство просмотра ассемблерного кода (Compiler Explorer / objdump и др.)

Порядок выполнения

1. Написать простую программу на языке C.

```c
#include <stdio.h>

int multiply(int a, int b) {
    return a * b;
}

int main() {
    int x = 5;
    int y = 10;
    int result;

    if (x < y) {
        result = multiply(x, y);
    } else {
        result = x + y;
    }

    printf("Результат: %d\n", result);

    return 0;
   }
```

2. Скомпилировать программу и получить ассемблерный код.

Компиляция программы

```
gcc -S program.c -o program.s


gcc -S -masm=intel program.c -o program_intel.s


gcc -c program.c -o program.o
objdump -d program.o
```

Ассемблерный код функции multiply (Intel синтаксис)

```
multiply:
    push    rbp
    mov     rbp, rsp
```

```
    mov     DWORD PTR [rbp-4], edi
    mov     DWORD PTR [rbp-8], esi
    mov     eax, DWORD PTR [rbp-4]
    imul    eax, DWORD PTR [rbp-8]
    pop     rbp
    ret
```

Ассемблерный код функции main (Intel синтаксис)

```
main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 32

    mov     DWORD PTR [rbp-4], 5
    mov     DWORD PTR [rbp-8], 10

    mov     eax, DWORD PTR [rbp-4]
    cmp     eax, DWORD PTR [rbp-8]
    jge     .L2

    # then-ветка (x < y)
    mov     edx, DWORD PTR [rbp-8]
    mov     eax, DWORD PTR [rbp-4]
    mov     esi, edx
    mov     edi, eax
    call    multiply
    mov     DWORD PTR [rbp-12], eax
    jmp     .L3
.L2:    # else-ветка (x >= y)
    mov     edx, DWORD PTR [rbp-4]
    mov     eax, DWORD PTR [rbp-8]
    add     eax, edx
    mov     DWORD PTR [rbp-12], eax

.L3:    # вывод результата
    mov     eax, DWORD PTR [rbp-12]
    mov     esi, eax
    lea     rdi, .LC0
    mov     eax, 0
    call    printf

    mov     eax, 0
    leave
    ret
```

3. Найти инструкции:

Инструкция MOV

Назначение: Копирование данных из источника в приёмник.

Формат	Описание	Пример из кода
`mov dst, src`	`dst = src`	`mov DWORD PTR [rbp-4], 5`
		`mov eax, DWORD PTR [rbp-4]`

Пример из кода

```
mov     DWORD PTR [rbp-4], 5
mov     eax, DWORD PTR [rbp-4]
mov     esi, edx
mov     eax, 0
```

Типы операндов:

- mov reg, reg — регистр → регистр
- mov reg, mem — память → регистр
- mov mem, reg — регистр → память
- mov reg, imm — непосредственное значение → регистр

Инструкция ADD

Назначение: Сложение двух операндов. Результат сохраняется в первый операнд.

Формат	Описание	Пример из кода
`add dst, src`	`dst = dst + src`	`add eax, edx`

Пример из программы

```
add     eax, edx
add     DWORD PTR [rbp-4], eax
add     DWORD PTR [rbp-8], 1
```

Родственные инструкции:

- sub — вычитание
- inc — увеличение на 1 (быстрее чем add reg, 1)
- dec — уменьшение на 1

Инструкция CALL

Назначение: Вызов функции (подпрограммы). Сохраняет адрес возврата в стек и передаёт управление.

Формат	Описание	Пример из кода
`call address`	переход к функции	`call multiply`
		`call printf`

Что происходит при CALL:

```
Перед вызовом:
    RIP = адрес инструкции CALL
    RSP = вершина стека

Шаг 1: RSP = RSP - 8           (выделение места в стеке)
Шаг 2: [RSP] = RIP + 5         (сохранение адреса возврата)
Шаг 3: RIP = адрес функции     (переход к функции)

После RET (возврата):
    RIP = [RSP]                (восстановление адреса)
    RSP = RSP + 8              (очистка стека)
```

Соглашение о вызовах (x86-64 System V):

Регистр	Назначение
RDI	1-й аргумент
RSI	2-й аргумент
RDX	3-й аргумент
RCX	4-й аргумент
R8	5-й аргумент
R9	6-й аргумент
RAX	возвращаемое значение

Инструкция JMP

Назначение: Переход на другой адрес (изменение потока выполнения).

Формат	Описание	Пример из кода
`jmp address`	безусловный переход	`jmp .L3`
`jge address`	переход если ≥	`jge .L2`
`jl address`	переход если <	`jl .L5`

Условные переходы (после CMP):

Инструкция	Условие	Значение
`je` / `jz`	равно	ZF = 1
`jne` / `jnz`	не равно	ZF = 0
`jg` / `jnle`	больше (со знаком)	ZF = 0 и SF = OF
`jge` / `jnl`	больше или равно	SF = OF
`jl` / `jnge`	меньше	SF ≠ OF
`jle` / `jng`	меньше или равно	ZF = 1 или SF ≠ OF
`ja` / `jnbe`	выше (без знака)	CF = 0 и ZF = 0
`jb` / `jnae`	ниже (без знака)	CF = 1

Пример из программы (условный оператор if-else):

```
mov     eax, DWORD PTR [rbp-4]
cmp     eax, DWORD PTR [rbp-8]
jge     .L2
# ... then-ветка ...
jmp     .L3
.L2:
# ... else-ветка ...
.L3:
# ... продолжение ...
```

4. Определить назначение каждой инструкции.

5. Описать процесс выполнения программы на уровне инструкций.

Жизненный цикл инструкции

Процессор выполняет каждую инструкцию в несколько этапов (конвейер):

```
+---------+     +---------+     +---------+     +---------+     +---------+
|  FETCH  | --> | DECODE  | --> |  EXEC   | --> | MEMORY  | --> | WRITEB  |
| (выборка)|    |(декодир.)|    |(исполн.)|    | (память) |    | (запись) |
+---------+     +---------+     +---------+     +---------+     +---------+
      |               |               |               |               |
      v               v               v               v               v
  Загрузка      Определение     Вычисление      Чтение/        Сохранение
  инструкции    типа и          результата      запись в       результата
  из памяти     операндов                       память         в регистр
```

Выводы

В ходе выполнения лабораторной работы были изучены основные машинные инструкции процессора на примере программы на языке C.

Основные результаты:

1. **Инструкция MOV** — фундаментальная инструкция для перемещения данных. Встречается чаще всего (более 50% всех инструкций в типичной программе). Используется для загрузки констант, передачи аргументов, сохранения результатов.
2. **Инструкция ADD** — выполняет арифметическое сложение. Может работать с регистрами, памятью и непосредственными значениями. В программе использована для вычисления суммы в else-ветке.
3. **Инструкция CALL** — обеспечивает механизм вызова функций. Сохраняет адрес возврата в стеке, что позволяет после ret вернуться в вызывающую функцию. Работает в паре с ret.
4. **Инструкция JMP** — управляет потоком выполнения. В форме условных переходов (JGE, JL, JE и др.) реализует конструкции if-else и циклы. Безусловный JMP используется для обхода else-ветки.

Процесс выполнения программы на уровне инструкций:

- Каждая инструкция кодируется в машинный код (от 1 до 15 байт)
- Процессор выполняет цикл: **FETCH → DECODE → EXEC → MEMORY → WRITEBACK**
- Регистры общего назначения (RAX, RBX, RCX, RDX, RSI, RDI, RSP, RBP) используются для быстрого хранения данных
- Стек используется для сохранения адресов возврата, локальных переменных и передачи аргументов (если их больше 6)
- Условные переходы зависят от флагов, установленных инструкцией CMP

Практическая значимость:

Понимание машинных инструкций необходимо для:

- Оптимизации производительности критического кода
- Отладки на низком уровне (reverse engineering)
- Написания высокоэффективных алгоритмов
- Понимания работы компилятора и связывания
- Изучения системных уязвимостей